

Data Structure

Level 1

Big $O(\log n)$: Logarithmic
Time Function

Data Structure

Level 1

Complexity of an Algorithm

Big O

$O(\log n)$

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing over a lighter, textured surface. In the lower portion of the image, there are dark silhouettes of several people, some standing and some appearing to be in a group, which adds a human element to the technical theme.

Data Structure

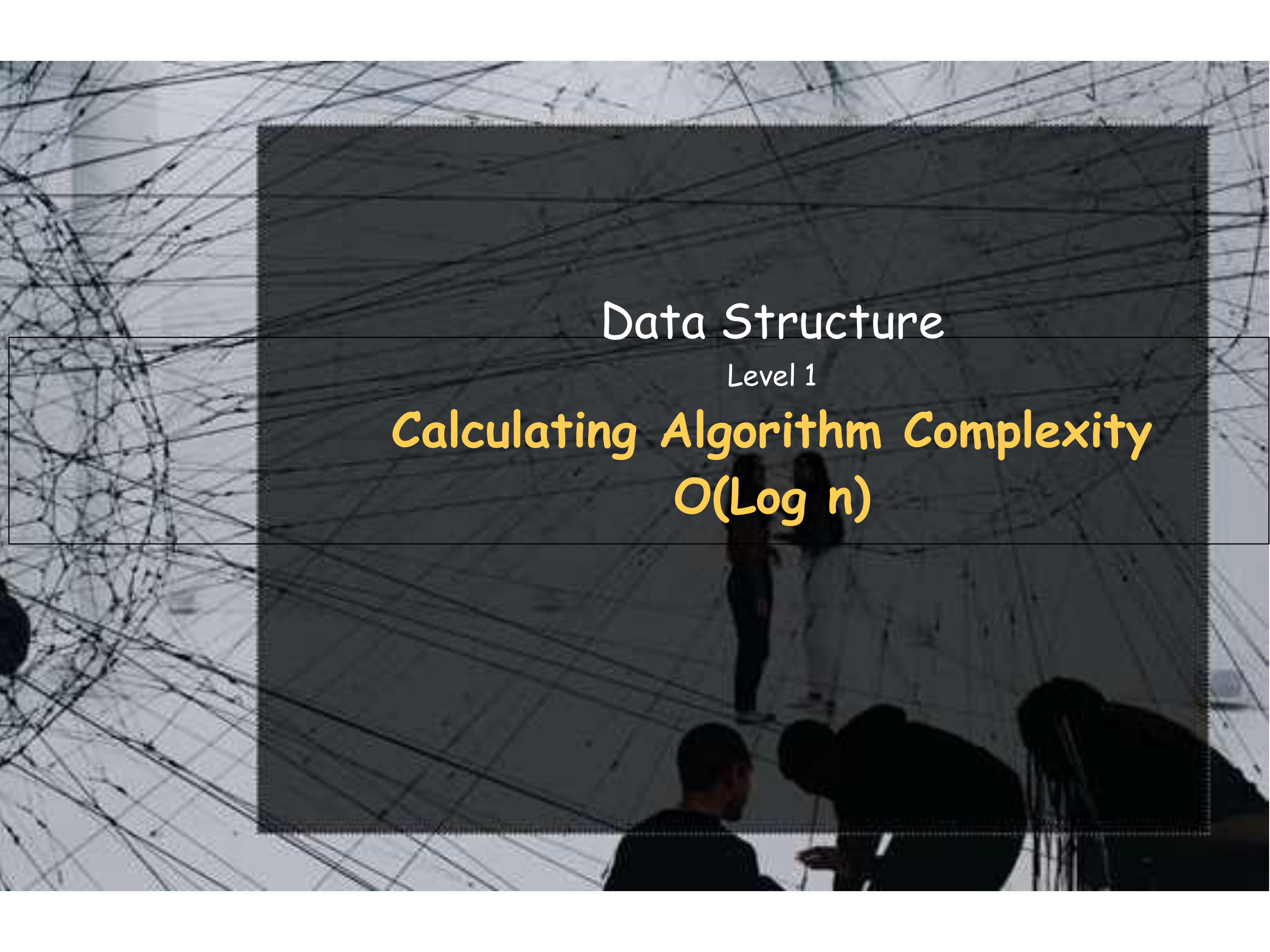
Level 1

What do we mean by log?

What do we mean by log?

$x^2 \rightarrow$ double itself

$\text{Log}2 \rightarrow$ half itself



Data Structure

Level 1

Calculating Algorithm Complexity $O(\log n)$

Calculating Algorithm Complexity:

```
void fun1(short n)
{
```

```
    short x = n ;
```

```
    while ( x > 0 )
```

```
    {
```

```
        x = x / 2 ;
```

```
        cout << x << endl ;
```

```
    }
```

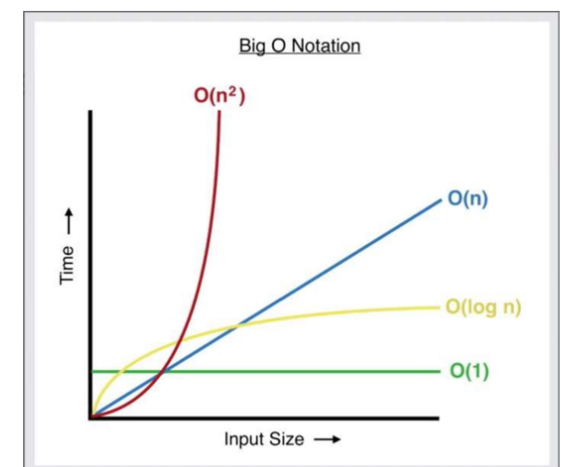
```
}
```

Number of Steps outside loop = 1

Number of Steps inside loop = 7

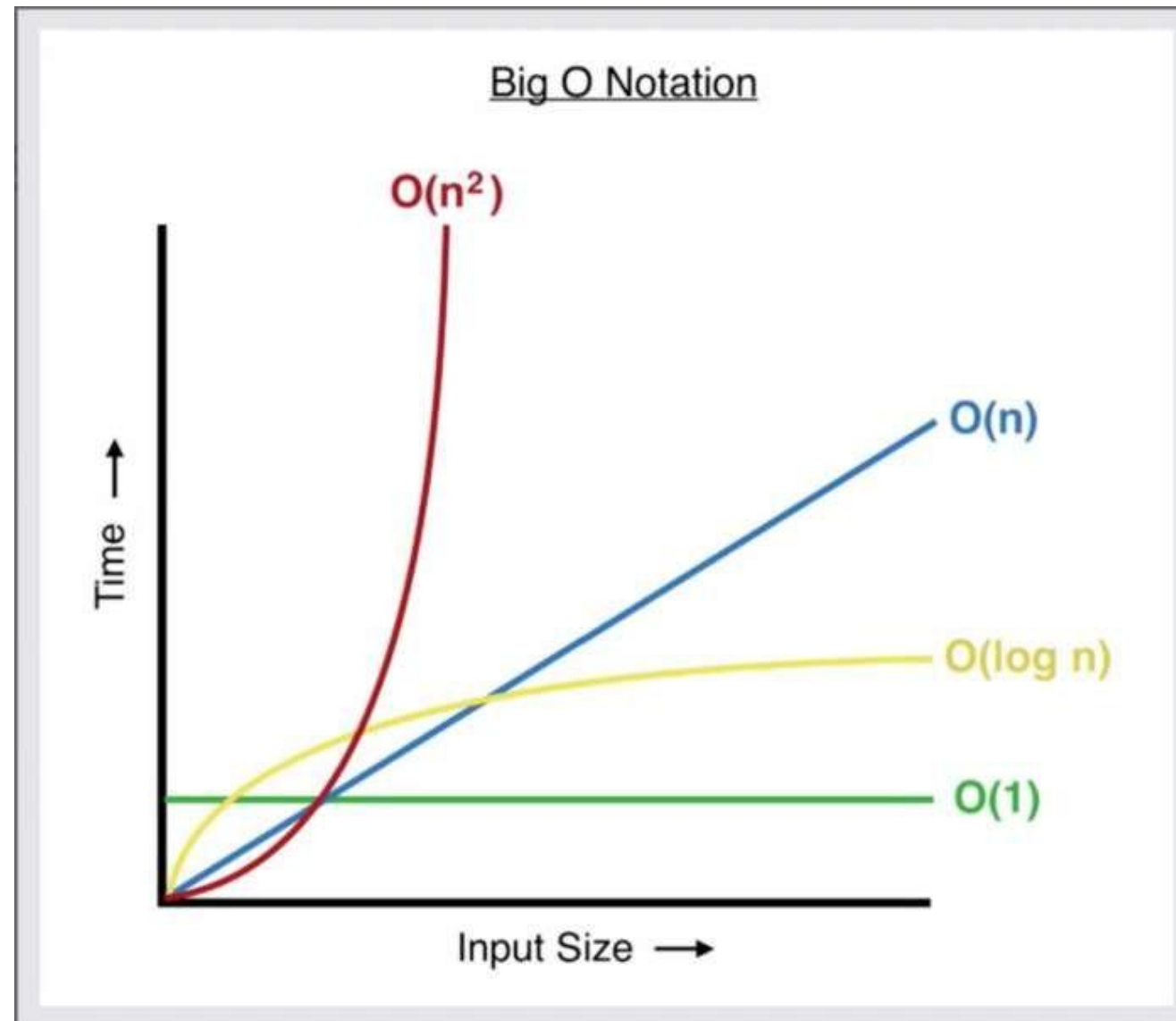
Big O =
 $7 * \log n$
 $\rightarrow \log n$

Big O =
 $1 + \log n$
 $\rightarrow \log n$



Depends on n size, and relation is logarithmic Time Function.

Which is faster?



Thank you 😊